

National University of Computer & Emerging Sciences, Karachi

Department of Computer Science

Fall-2025

CS2009: Design and Analysis of Algorithms

Assignment 5

Due Date: 7th December 2025

20% penalty for 1 day late

40% penalty for 2 days late

Submission is not allowed afterward

Max Marks: 100 Marks

Question#01: Watch the video lecture on [P, NP, NP-hard, and NP-complete problems](#). And write the answer to the following question in your own words. **[70 Marks]**

- a) What is meant by P and NP Problems? Explain $P = NP$.

P is the class of decision problems (yes/no questions) that can be solved by a deterministic Turing machine in time polynomial in the input size n . Informally: problems for which there exists an algorithm whose running time is $O(n^k)$ for some constant k . Example: sorting-related decision tasks, connectivity in graphs, shortest path with nonnegative weights (Dijkstra), etc

NP is the class of decision problems for which a proposed solution (a certificate) can be verified in polynomial time by a deterministic algorithm. Equivalently, problems solvable by a nondeterministic Turing machine in polynomial time. Example: given a graph and a set of k vertices, verifying that set is a vertex cover is polynomial-time.

This is the famous open question: are all problems whose solutions can be verified in polynomial time also solvable in polynomial time? Formally, is P equal to NP ? If $P=NP$, then every problem that has polynomial-time verifiers would also have polynomial-time algorithms to find solutions. Most researchers believe $P \neq NP$, but it remains unproven.

- b) Why it is important to find approximate solutions for NP-Complete Problems.

Intractability: NP-complete problems are believed not to have polynomial-time exact algorithms (unless $P=NP$). For moderate-to-large input sizes, exact algorithms are often impractical.

Practical needs: Many real-world applications require solutions quickly (scheduling, routing, packing). Approximation algorithms give "good" solutions in polynomial time.

Guaranteed quality: Approximation algorithms often come with provable bounds (e.g., within factor 2 of optimal). That gives predictable performance.

Tradeoff speed vs quality: They enable trading off running time for closeness to optimum — often the only feasible approach for large instances.

Heuristics without guarantees are risky: Approximation algorithms provide performance guarantees missing from many heuristics.

- c) What is the difference between NP-Hard and weakly NP-hard class problems?

NP-hard: A problem is NP-hard if every problem in NP can be polynomial-time reduced to it. NP-hard problems may be decision problems, optimization problems, or even undecidable problems. Being NP-hard does not require membership in NP (i.e., they may not have polynomial-time verifiers).

Weakly NP-hard (or weakly NP-complete for decision versions): This typically applies to numerical/optimization problems where the hardness depends on numeric values encoded in binary. A problem is weakly NP-hard if it is NP-hard in the ordinary encoding, but there exists a pseudo-polynomial time algorithm (running time polynomial in the numeric value of input numbers, not in the number of bits). Example: Subset Sum and Knapsack are weakly NP-hard because dynamic programming solves them in time polynomial in the numeric sum (pseudo-polynomial).

Strongly NP-hard: No pseudo-polynomial algorithm is likely to exist; hardness remains even when numbers are small (i.e., problem is hard even under unary encoding).

- d) What is the 3-SAT problem?

3-SAT (3-satisfiability) is a Boolean satisfiability decision problem in which the formula is in conjunctive normal form (CNF) and each clause has exactly three literals. Given a Boolean formula $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ where each clause C_j is the OR of exactly three literals (a literal is a variable or its negation), the question is: does there exist a truth assignment to the variables that makes the entire formula ϕ true? 3-SAT is a canonical NP-complete problem.

- e) What is meant by NP-complete problems? How can we prove a problem is NP-complete? Explain with an example.

NP-complete: A decision problem is NP-complete if:

It is in NP (solutions can be verified in polynomial time), and

It is NP-hard (every problem in NP can be polynomial-time reduced to it).

How to prove a problem is NP-complete (standard approach):

Show membership in NP: Give a polynomial-time verification algorithm for a candidate solution.

Show NP-hardness: Pick a known NP-complete problem AAA (commonly 3-SAT, SAT, or another established NP-complete) and give a polynomial-time transformation fff that maps any instance xxx of AAA to an instance f(x)f(x)f(x) of your problem such that xxx is a yes-instance iff f(x)f(x)f(x) is a yes-instance. This proves every instance of AAA reduces to your problem, hence your problem is at least as hard as AAA.

Example (sketch): Prove CLIQUE is NP-complete.

- CLIQUE: Given graph G and integer k, does G contain a clique of size $\geq k$?
- **In NP?** Yes — certificate is the vertex set; verify all pairs are connected in $O(k^2)$.
- **NP-hard?** Reduce from 3-SAT (or from an already known NP-complete problem like INDEPENDENT-SET). Provide a polynomial-time construction that maps a 3-SAT formula to a graph where cliques correspond to satisfying assignments (standard textbook reductions exist). Thus, CLIQUE is NP-complete.

- f) What is Reduction?

A **reduction** from problem A to problem B is a way to transform any instance of A into an instance of BBB such that the answer to the transformed instance of BBB corresponds to the answer to the original instance of A. In complexity theory we usually require reductions to be computable in polynomial time (polynomial-time many-one reductions). If $A \leq_p B$ (A reduces to B in polynomial time) and B has a polynomial-time algorithm, then A also has a polynomial-time algorithm. Reductions are the main tool for proving hardness.

- g) A problem that is solvable in time complexity of $T(n)=3 \cdot n^n$ and space complexity of $S(n)=n^2$ and it can be validated in $T(n)=2^n$ time. Is it an NP-Complete or NP-Hard? Explain.

To be in **NP**, a problem must have a verifier that runs in **polynomial time**. Here validation (verification) time is 2^n , which is exponential not polynomial. So, under that interpretation the problem is **not in NP**.

Since the problem is solvable (decidable) in time $3 \cdot n$ (which is super-polynomial), it could still be hard.

A problem that is not in NP cannot be **NP-complete** (by definition NP-complete problems must lie in NP). However, it can be **NP-hard** because NP-hardness only requires that all problems in NP reduce to it; there is no requirement that it be in NP.

Conclusion: The problem is **not NP-complete** (unless the verification time was mis-stated and is actually polynomial). It **could** be **NP-hard** (and plausibly is), but we cannot conclude NP-completeness because it fails the polynomial-time verifier requirement. In short: **NP-Hard (possible) but NOT NP-Complete** given the exponential verification time.

Question#02: Consider the following APPROX-VERTEX-COVER algorithm. Proof that this algorithm is a 2approximation method for VERTEX-COVER.

[15 Marks]

```
APPROX-VERTEX-COVER(G)
C = ∅; E' = G.E;
while(E' ≠ ∅){
  Randomly choose a edge (u,v) in E', put u and v into C; Remove all the edges
  that covered by u or v from E'
}
Return C;
```

Solution:

Let the algorithm select edges $E_1, E_2, \dots, E_k$ in the while loop (each selection removes all edges incident to its two endpoints). The algorithm adds both endpoints of each selected edge to C, so $|C|=2k$.

Observations:

- The selected edges E_i are pairwise edge-disjoint (no two of the chosen edges share a vertex). Reason: when an edge $E_i=(u,v)$ is chosen, we remove all edges incident on u or v from E' , so subsequent chosen edges cannot share u or v.
- Any vertex cover must cover every chosen edge E_i . For each chosen edge, at least one endpoint must belong to any vertex cover. Since the edges are disjoint, a single vertex cannot cover two different chosen edges (because edges are disjoint). Therefore, any vertex cover must include at least one distinct vertex per chosen edge, so $|C^*| \geq k$.

Combine the two facts:

$$|C|=2k \leq 2 \cdot |C^*|.$$

Thus the algorithm is a 2-approximation.

Question#03: An Instance (X, F) of the set-covering problem consists of a finite set X and a family F of the subset of X , such that every element of X belongs to at least one subset of F :

$$X = \bigcup_{S \in F} S$$

We say that a subset $S \in F$ covers all elements in X . Our goal is to find a minimum size subset $C \subseteq F$ whose members cover all of X .

$$X = \bigcup_{S \in C} S$$

Algorithm 1: GREEDY-SET-COVER (X, F)

```

1  $U \leftarrow X$ 
2  $C \leftarrow \emptyset$ 
3 While  $U \neq \emptyset$ 
4   do select an  $S \in F$  that maximizes  $|S \cap U|$ 
5      $U \leftarrow U - S$ 
6      $C \leftarrow C \cup \{S\}$ 
7 return  $C$ 

```

Consider each of the following words as a set of letters: {arid, dash, drain, heard, lost, nose, shun, slate, snare, thread}. Show which set cover GREEDY-SET-COVER produces when we break ties in favor of the word that appears first in the dictionary. **[15 Marks]**

Solution:

We treat each word as a set of its letters. The family F consists of sets corresponding to these words:

Words and their letter sets:

- **arid** $\rightarrow \{a, d, i, r\}$
- **dash** $\rightarrow \{d, a, s, h\}$
- **drain** $\rightarrow \{d, r, a, i, n\}$
- **heard** $\rightarrow \{h, e, a, r, d\}$
- **lost** $\rightarrow \{l, o, s, t\}$
- **nose** $\rightarrow \{n, o, s, e\}$
- **shun** $\rightarrow \{s, h, u, n\}$
- **slate** $\rightarrow \{s, l, a, t, e\}$
- **snare** $\rightarrow \{s, n, a, r, e\}$
- **thread** $\rightarrow \{t, h, r, e, a, d\}$

First compute the **universe** X (all letters appearing):

$X = \{a, d, e, h, i, l, n, o, r, s, t, u\}$

(12 letters in total.)

We must run the GREEDY-SET-COVER algorithm that repeatedly picks the set that covers the largest number of currently uncovered elements. Ties are broken in favor of the word that appears first in the dictionary. (Dictionary order used: arid, dash, drain, heard, lost, nose, shun, slate, snare, thread.)

I show the greedy steps (counts are number of *new* letters the set would cover):

Initial uncovered set = X .

1. Evaluate new-elements for every set:

- arid: 4 new
- dash: 4 new
- drain: 5 new
- heard: 5 new
- lost: 4 new
- nose: 4 new
- shun: 4 new
- slate: 5 new
- snare: 5 new
- thread: 6 new

Max = 6: choose **thread** (covers {t,h,r,e,a,d})

Uncovered after pick: {i,l,n,o,s,u}

2. Re-evaluate new elements (on remaining sets):

- arid: covers {i} → 1 new
- dash: covers {s} → 1 new (d,a,h already covered)
- drain: covers {i,n} → 2 new
- heard: covers {} → 0 new (all letters already covered by thread)
- lost: covers {l,o,s} → 3 new
- nose: covers {n,o,s} → 3 new
- shun: covers {n,s,u} → 3 new
- slate: covers {l,s} → 2 new (a,e,t already covered)
- snare: covers {n,s} → 2 new

Best new coverage = 3 (lost, nose, shun). Tie-breaker: pick the one appearing first in dictionary among them. Dictionary order of those three: **lost** (comes before nose, shun).

Pick lost (covers {l,o,s,t})

Uncovered now: {i,n,u}

3. Re-evaluate:

- arid: {i} → 1
- dash: {} → 0
- drain: {i,n} → 2
- heard: {} → 0
- nose: {n} → 1
- shun: {n,u} → 2
- slate: {} → 0
- snare: {n} → 1

Best new coverage = 2 (drain and shun). Tie-breaker (dictionary order): **drain** comes before shun.

Pick drain (covers {d,r,a,i,n})

Uncovered now: {u}

4. Re-evaluate:

- Only set that covers u is **shun** ($\{s, h, u, n\}$) $\rightarrow$ covers $\{u\}$ (1). All others cover 0 new elements.

Pick shun. Uncovered becomes empty.

Greedy solution (in selection order):

thread, lost, drain, shun.

The set cover returned by GREEDY-SET-COVER:

$\{\text{thread, lost, drain, shun}\}$.

Verify coverage:

Union of their letters =

$\text{thread} \cup \text{lost} \cup \text{drain} \cup \text{shun} = \{a, d, e, h, r, t\} \cup \{l, o, s, t\} \cup \{d, r, a, i, n\} \cup \{s, h, u, n\} = \{a, d, e, h, i, l, n, o, r, s, t, u\} = X$.

So all 12 letters are covered.

****Best of luck****